
T.R.A.I.L

Terrain Rando Alerte Info Live

Rapport de projet — Module TI616 Numérique Durable

Établissement	EFREI Paris
Module	TI616 – Numérique Durable
Coordinateur	Yvan GUIFO
Année	2025–2026
Livrable	Rapport final
Date	Mai 2026

Membres du groupe :

Prénom Nom	Rôle principal
Loïc Anglard	Développeur
Esteban Batard-Plaza	Front-end / UI
Lilian Cao	Test / Intégration
Maelle Chollet	Analyste carbone
Mathieu Danthine	Développeur

Table des matières

1	Présentation du projet	3
1.1	Le problème de départ	3
1.2	La solution	3
1.3	Périmètre du MVP	3
1.4	Utilisateurs cibles	3
1.5	Contraintes Green IT retenues dès la conception	3
2	Architecture et conception	4
2.1	Architecture globale	4
2.2	Structure du projet	4
2.3	Organisation backend / frontend	5
2.4	Routes Flask principales	6
2.5	Diagrammes UML	6
2.6	Schéma logique de la base de données	8
3	Choix technologiques justifiés	9
3.1	Pourquoi Flask plutôt que Django	9
3.2	Pourquoi éviter un framework front-end	9
3.3	SQLite en local et PostgreSQL en ligne	9
4	Implémentation	10
4.1	Fonctionnalités développées	10
4.2	Authentification	10
4.3	Ajout d'un sentier	11
4.4	Détail d'un sentier	11
4.5	Dépôt d'un rapport	12
4.6	Expiration automatique des rapports	13
4.7	Profil utilisateur	14
4.8	Sécurité et contrôle des accès	14
5	Analyse de l'empreinte carbone	15
5.1	Choix de sobriété	15
5.2	Encart Green IT	15
5.3	Résultat EcoIndex	15
5.4	Ce qui a été exclu volontairement	16
6	Tests et validation	17
6.1	Scénarios de tests fonctionnels	17
6.2	Validation visuelle	17
6.3	Résultats Lighthouse	17
7	Organisation de l'équipe et collaboration	19
7.1	Répartition des tâches	19
7.2	Collaboration	19
8	Discussion et conclusion	20
8.1	Ce qui a bien fonctionné	20
8.2	Difficultés rencontrées	20
8.3	Compromis entre fonctionnalités et sobriété	20
8.4	Pistes d'amélioration	20
8.5	Conclusion	20

A Backlog complet	21
B Schéma SQL simplifié	21
C Captures complémentaires	22

1. Présentation du projet

1.1 Le problème de départ

Avant une sortie trail, VTT ou randonnée, la question revient souvent : ce sentier est-il praticable ce week-end ? Les informations disponibles en ligne sont parfois trop anciennes, dispersées entre forums, groupes de réseaux sociaux ou blogs de clubs locaux. Rien n'est toujours centralisé, rien n'est vraiment structuré, et surtout les données ne sont pas toujours récentes.

1.2 La solution

T.R.A.I.L (Terrain Rando Alerte Info Live) est une plateforme collaborative permettant aux pratiquants de trail, VTT, randonnée ou ski de randonnée de consulter et partager l'état réel des itinéraires.

Le principe central est l'**expiration automatique des rapports à 7 jours**. Un rapport trop ancien est masqué : l'application privilégie les informations récentes et utiles. C'est à la fois une décision fonctionnelle et une décision liée à la sobriété numérique, car il est inutile d'afficher indéfiniment des données devenues obsolètes.

1.3 Périmètre du MVP

- Authentification : inscription, connexion, déconnexion.
- Gestion du profil utilisateur : modification, suppression de compte avec cascade.
- Liste des sentiers avec filtres par région et difficulté.
- Page de détail d'un sentier avec ses rapports actifs.
- Ajout, modification et suppression de sentiers.
- Dépôt, modification et suppression de rapports de conditions.
- Expiration automatique des rapports à 7 jours.
- Encart Green IT en pied de page.
- Mode clair / sombre.

Ont été volontairement exclus du MVP : notifications e-mail, carte interactive intégrée, export CSV, API publique JSON et fonctionnalités sociales avancées.

1.4 Utilisateurs cibles

Les utilisateurs visés sont principalement des pratiquants d'activités outdoor : randonneurs, coureurs trail, VTTistes ou skieurs de randonnée. Le besoin principal est de pouvoir obtenir rapidement une information récente avant une sortie, notamment en cas de météo changeante, de boue, de neige, de travaux ou d'obstacles sur le parcours.

1.5 Contraintes Green IT retenues dès la conception

Poids de page inférieur à 200 Ko — Moins de 10 requêtes HTTP par page — Aucune ressource externe inutile — Pas de CDN — Pas de police externe — Pas de tracker — Données principalement textuelles — Interface HTML/CSS native sans framework front lourd.

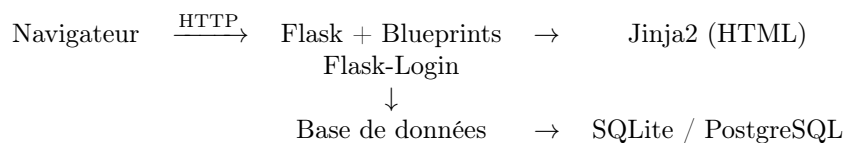


FIGURE 1 – Page d’accueil de T.R.A.I.L

2. Architecture et conception

2.1 Architecture globale

T.R.A.I.L repose sur une architecture web classique côté serveur : Flask reçoit les requêtes HTTP, Jinja2 produit le HTML, et la base de données stocke les entités principales. Il n’y a pas de framework JavaScript de rendu côté client, ce qui limite fortement le poids des pages.



2.2 Structure du projet

Le projet est organisé autour d’une séparation entre backend, frontend et base de données. Cette organisation facilite la maintenance et permet de retrouver rapidement les fichiers liés à une fonctionnalité.

T.R.A.I.L — Diagramme de classes

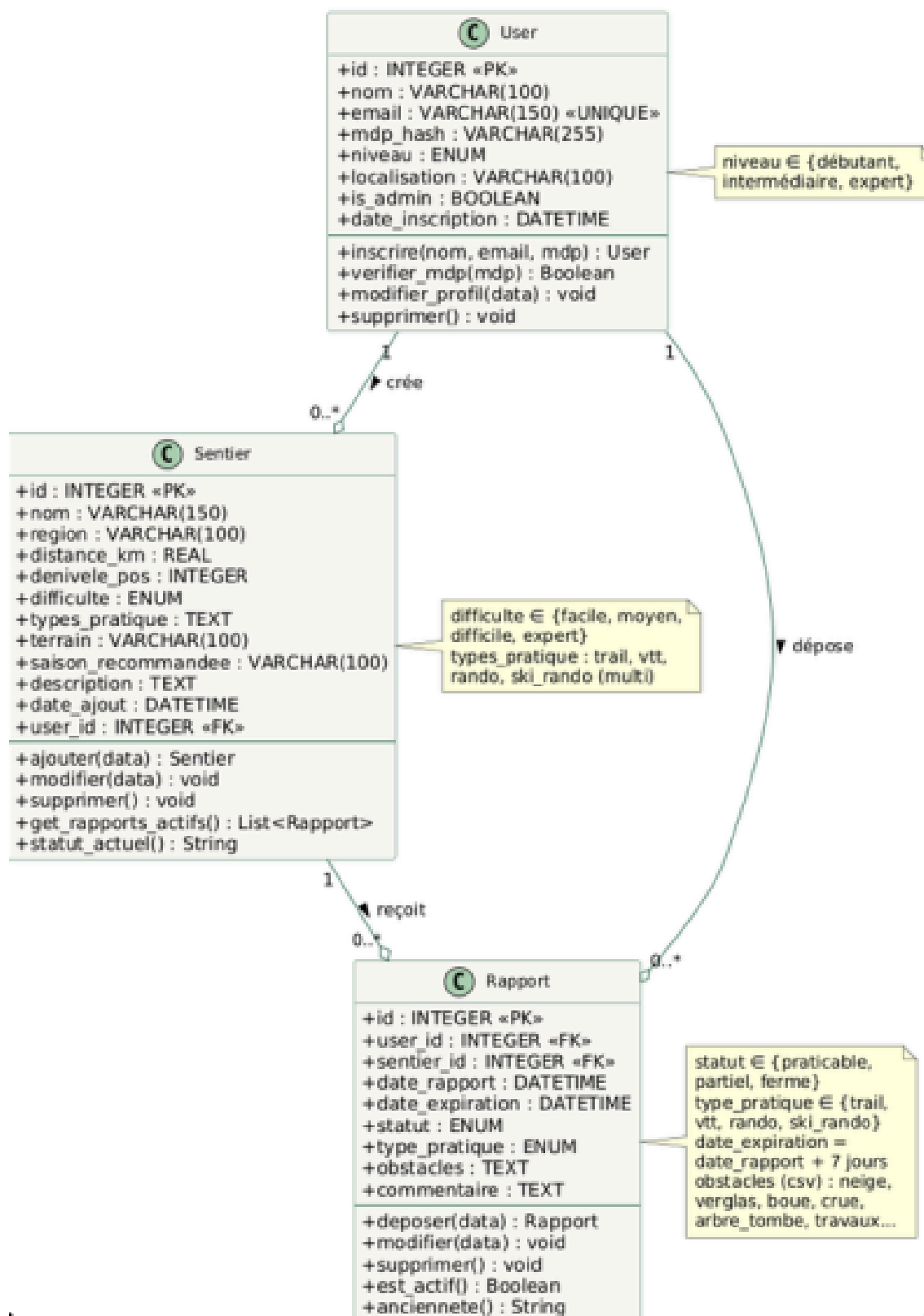


FIGURE 2 — Arborescence du projet T.R.A.I.L

2.3 Organisation backend / frontend

Le backend est divisé en routes Flask :

- auth.py : inscription, connexion, déconnexion.

- `users.py` : profil utilisateur et gestion du compte.
- `sentiers.py` : liste, création, modification et suppression de sentiers.
- `rapports.py` : dépôt, modification et suppression de rapports de conditions.

Le frontend repose sur des templates Jinja2 organisés par domaine fonctionnel. Le style visuel est centralisé dans un fichier CSS unique, ce qui évite de multiplier les ressources chargées par le navigateur.

2.4 Routes Flask principales

Méthode	Route	Accès	Description
GET	/	Public	Accueil, présentation du service
GET/POST	/auth/inscription	Public	Création de compte
GET/POST	/auth/connexion	Public	Connexion utilisateur
GET	/auth/deconnexion	Connecté	Déconnexion
GET	/sentiers/	Public	Liste des sentiers
GET	/sentiers/<id>	Public	Détail d'un sentier
GET/POST	/sentiers/nouveau	Connecté	Ajout d'un sentier
GET/POST	/rapports/nouveau	Connecté	Dépôt d'un rapport
GET/POST	/utilisateurs/profil	Connecté	Profil utilisateur

2.5 Diagrammes UML

Les diagrammes UML permettent de représenter la conception du projet avant son implémentation. Ils montrent les interactions principales entre les utilisateurs, les sentiers et les rapports de conditions.

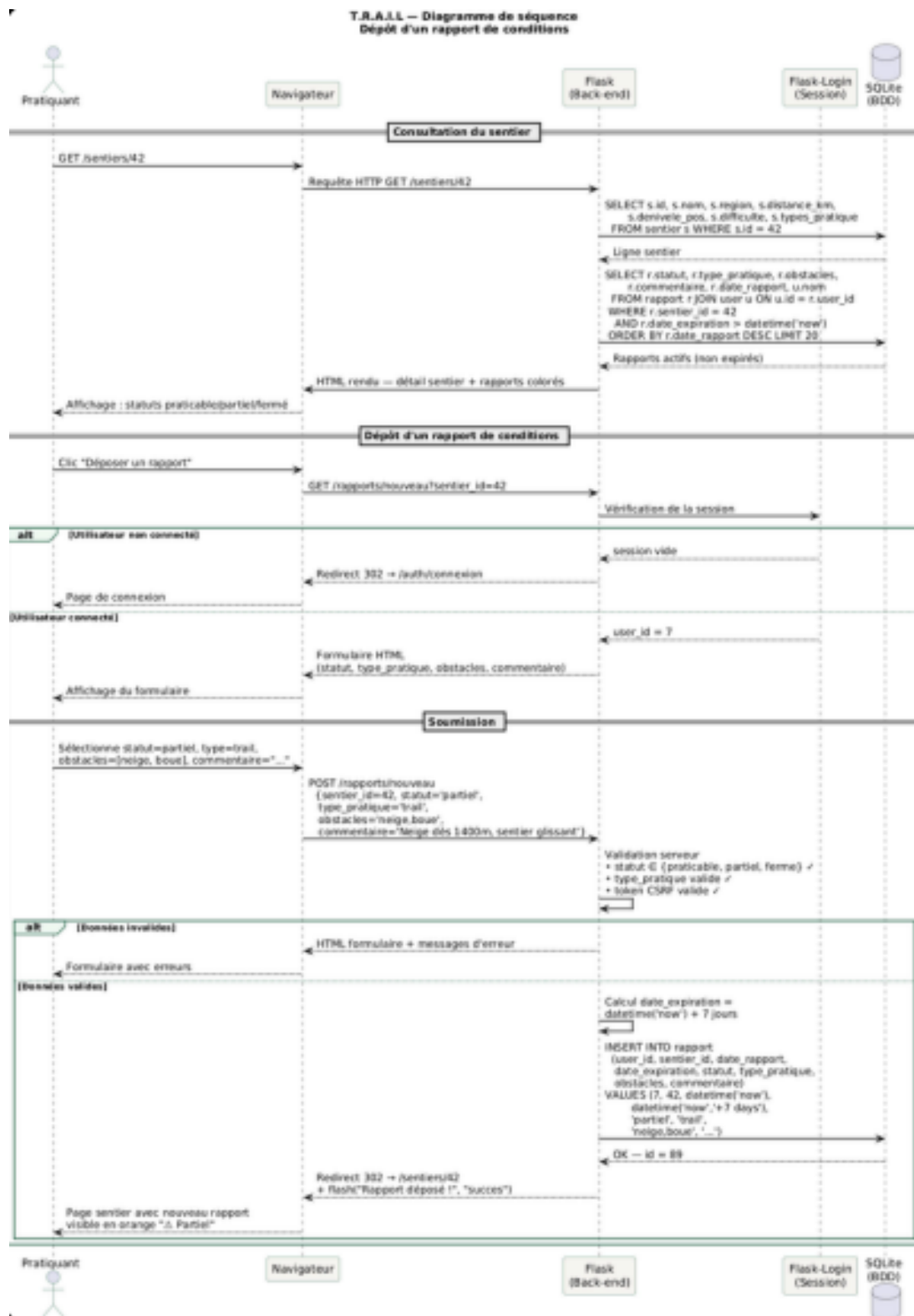


FIGURE 3 — Diagramme de séquence : dépôt d'un rapport de conditions

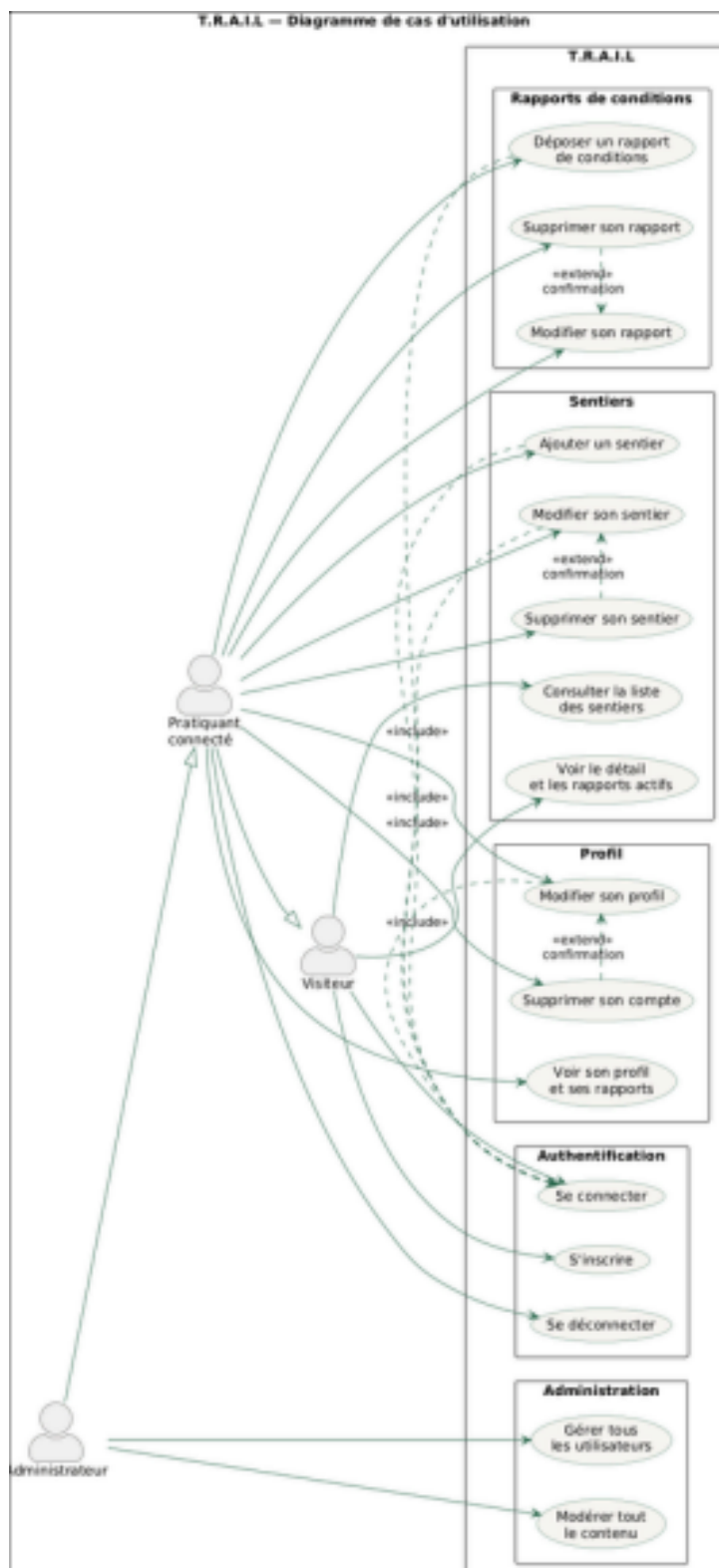


FIGURE 4 – Diagramme de classes de T.R.A.I.L

2.6 Schéma logique de la base de données

Le modèle repose sur trois entités principales : **user**, **sentier** et **rapport**. Un utilisateur peut créer plusieurs sentiers et déposer plusieurs rapports. Un sentier peut recevoir plusieurs rapports de conditions.

Table	Rôle
user	Stocke les comptes utilisateurs, le mot de passe hashé, le niveau et la localisation.
sentier	Stocke les informations principales d'un itinéraire : nom, région, distance, dénivelé, difficulté, pratiques.
rapport	Stocke les signalements communautaires : statut, activité, obstacles, commentaire et date d'expiration.

3. Choix technologiques justifiés

Technologie	Usage	Justification technique	Intérêt Green IT
Flask	Back-end	Framework léger, adapté à un MVP avec routes simples.	Moins de dépendances et faible surcharge.
SQLite	Base locale	Simple à utiliser en développement, sans serveur dédié.	Peu de ressources nécessaires.
PostgreSQL / Supabase	Base externe envisagée	Permet une persistance réelle en ligne.	Infrastructure mutualisée.
Jinja2	Templates	Génération HTML côté serveur.	Pas de rendu JavaScript lourd côté client.
HTML/CSS natif	Interface	Maîtrise complète du style et du poids.	Pas de framework front chargé inutilement.
JavaScript minimal	Interactions simples	Utilisé uniquement pour quelques comportements utiles.	Pas de bundle JS lourd.
Flask-Login	Sessions	Gestion simple de l'authentification.	Évite une architecture plus complexe.
bcrypt	Sécurité MDP	Hashage robuste des mots de passe.	Sécurité sans service externe.
Flask-WTF	Formulaires	Validation et protection CSRF.	Limite les erreurs et requêtes invalides.

3.1 Pourquoi Flask plutôt que Django

Flask est mieux adapté à ce périmètre. Django aurait apporté un ORM, un panneau d'administration et beaucoup de fonctionnalités qui n'étaient pas nécessaires pour notre MVP. Avec Flask, nous avons pu construire une architecture plus légère et plus facile à comprendre.

3.2 Pourquoi éviter un framework front-end

Nous avons volontairement évité React, Vue, Bootstrap ou Tailwind. Pour une application essentiellement composée de formulaires, listes et pages de détails, ces outils auraient ajouté du poids et de la complexité. Le HTML généré par Jinja2 et le CSS natif suffisent pour répondre au besoin.

3.3 SQLite en local et PostgreSQL en ligne

SQLite est très pratique pour développer et tester localement. En revanche, pour un déploiement sur une plateforme serverless, il faut une base de données persistante externe. PostgreSQL via Supabase a donc été envisagé comme évolution de production.

4. Implémentation

4.1 Fonctionnalités développées

Les principales fonctionnalités développées sont :

- création de compte et connexion ;
- gestion du profil utilisateur ;
- ajout et consultation de sentiers ;
- dépôt de rapports de conditions ;
- affichage de statuts colorés ;
- filtrage des sentiers ;
- expiration automatique des rapports ;
- interface claire/sombre ;
- encart Green IT.

● Praticable Tour du Lac Blanc Haute-Savoie DISTANCE DÉNIVELÉ 11.8 km +890 m Voir les conditions →	● Partiel Crêtes de la Chartreuse Isère DISTANCE DÉNIVELÉ 18.4 km +1260 m Voir les conditions →
● Praticable Forêt de Fontainebleau Seine-et-Marne DISTANCE DÉNIVELÉ 9.6 km +180 m Voir les conditions →	● Fermé Belvédère des Aigles Savoie DISTANCE DÉNIVELÉ 14.2 km +980 m Voir les conditions →
● Praticable Lac des Miroirs Hautes-Alpes DISTANCE DÉNIVELÉ	● Partiel Traversée des Balcons Drôme DISTANCE DÉNIVELÉ

FIGURE 5 — Liste des sentiers avec statuts et niveaux de difficulté

4.2 Authentification

L'inscription hache le mot de passe avec bcrypt avant stockage. Flask-Login permet ensuite de conserver l'utilisateur connecté via une session.

```

1 mdp_hash = bcrypt.hashpw(
2     mot_de_passe.encode("utf-8"),
3     bcrypt.gensalt()
4 ).decode("utf-8")
5

```

```
6 db.execute(  
7     """  
8     INSERT INTO user (nom, email, mdp_hash, niveau, localisation)  
9     VALUES (?, ?, ?, ?, ?)  
10    """  
11    (nom, email, mdp_hash, niveau, localisation)  
12 )  
13 db.commit()
```

Listing 1 – Exemple simplifié d’inscription avec hachage bcrypt

4.3 Ajout d’un sentier

L’ajout d’un sentier est réservé aux utilisateurs connectés. Le formulaire récupère les informations principales : nom, région, distance, dénivelé, difficulté, type de pratique, terrain, saison et description.

```
1 db.execute(  
2     """  
3     INSERT INTO sentier  
4     (nom, region, distance_km, denivele_pos, difficulte,  
5     types_pratique, terrain, saison_recommandee, description, user_id)  
6     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)  
7     """  
8     (nom, region, distance, denivele, difficulte,  
9     types_pratique, terrain, saison, description, current_user.id)  
10 )  
11 db.commit()
```

Listing 2 – Insertion simplifiée d’un sentier en base

4.4 Détail d’un sentier

La page détail rassemble les informations techniques du sentier et les rapports déposés par la communauté.

[← Retour à l'exploration](#)

Signaler l'état
 Modifier
 Supprimer

Tour du Lac Blanc

Haute-Savoie · Ajouté par @Alice Martin

DISTANCE
11.8 km

DÉNIVELÉ POSITIF
+890 m

NIVEAU DE DIFFICULTÉ
 MOYEN

Informations techniques

Type de terrain

Période conseillée

Activités adaptées

Alpin

Juin à Octobre

TRAIL
 RANDO

Description

Boucle panoramique très fréquentée.

Rapports de la communauté

2 rapport(s) actif(s)

● Praticable
 TRAIL

@Alice Martin
 2026-05-06

OBSTACLES SIGNALÉS : Sec · Propre

"Très bon état, quelques portions humides au lever du jour."

Modifier Supprimer

● Partiel
 TRAIL

@Nadia Laurent
 2026-05-02

OBSTACLES SIGNALÉS :

FIGURE 6 — Page détail d'un sentier avec les rapports communautaires

4.5 Dépôt d'un rapport

Le dépôt d'un rapport permet d'indiquer si un sentier est praticable, partiellement praticable ou fermé. L'utilisateur peut aussi préciser les obstacles rencontrés et ajouter un commentaire.

Déposer un rapport

Partagez l'état du terrain avec la communauté

SENTIER CONCERNÉ *

Tour du Lac Blanc (Haute-Savoie) ▼

ÉTAT ACTUEL DU SENTIER *

☒ **PRATICABLE** CONDITIONS IDÉALES

☐ **PARTIEL** PASSAGES DÉLICATS

☐ **FERMÉ** DANGERS / TRAVAUX

VOTRE ACTIVITÉ *

Trail ▼

OBSTACLES RENCONTRÉS

☐ NEIGE
 ☐ BOUE
 ☐ VERGLAS
 ☐ ARBRE TOMBE
 ☐ CRUE
 ☐ TRAVAUX
 ☐ AUTRE

PRÉCISIONS ET OBSERVATIONS

Ex: Neige à partir de 1500m, arbres en travers du sentier sur la portion haute...

Ce signalement sera public et expirera automatiquement après 7 jours pour garantir la fraîcheur de l'information.

Déposer le rapport

Annuler

FIGURE 7 – Formulaire de dépôt d'un rapport de conditions

4.6 Expiration automatique des rapports

Lors de la création d'un rapport, une date d'expiration est calculée automatiquement. Cela permet de conserver uniquement des informations récentes à l'affichage.

```

1 date_rapport = datetime.utcnow()
2 date_expiration = date_rapport + timedelta(days=7)
3

```

```

4 db.execute(
5     """
6     INSERT INTO rapport
7     (user_id, sentier_id, date_rapport, date_expiration,
8     statut, type_pratique, obstacles, commentaire)
9     VALUES (?, ?, ?, ?, ?, ?, ?, ?)
10    """
11    (current_user.id, sentier_id, date_rapport, date_expiration,
12     statut, type_pratique, obstacles, commentaire)
13 )
14 db.commit()
  
```

Listing 3 – Calcul de la date d'expiration d'un rapport

4.7 Profil utilisateur

La page profil permet de modifier ses informations, de consulter ses derniers rapports et de retrouver les sentiers ajoutés.

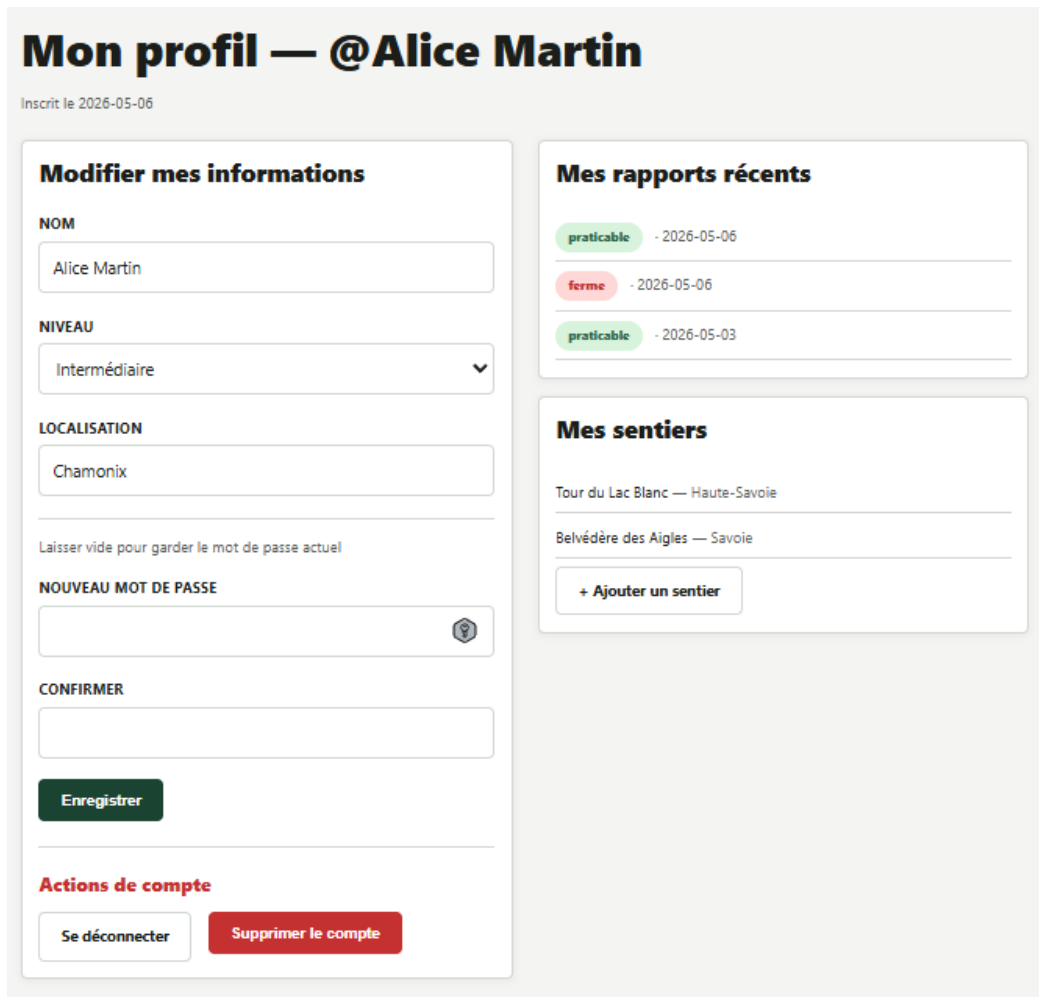


FIGURE 8 – Page profil utilisateur

4.8 Sécurité et contrôle des accès

Les routes privées utilisent `@login_required`. Les actions de modification ou suppression vérifient aussi que l'utilisateur est bien propriétaire de la ressource.

```

1 if sentier["user_id"] != current_user.id and not current_user.is_admin:
2     abort(403)
  
```

Listing 4 – Contrôle propriétaire simplifié

5. Analyse de l’empreinte carbone

5.1 Choix de sobriété

- **Pas de framework CSS externe.** Le style est écrit à la main dans un fichier CSS unique.
- **Pas de police externe.** L’application utilise une police système.
- **Pas de JavaScript lourd.** Le rendu se fait côté serveur avec Jinja2.
- **Données textuelles uniquement.** Le projet exclut les images uploadées, vidéos ou traces GPX.
- **Expiration des rapports.** Les informations obsolètes ne sont plus mises en avant.
- **Pagination et filtres.** Les listes restent limitées et lisibles.

5.2 Encart Green IT

L’application affiche un encart Green IT indiquant des mesures locales comme le poids transféré, le nombre de requêtes HTTP ou une estimation indicative de CO₂.

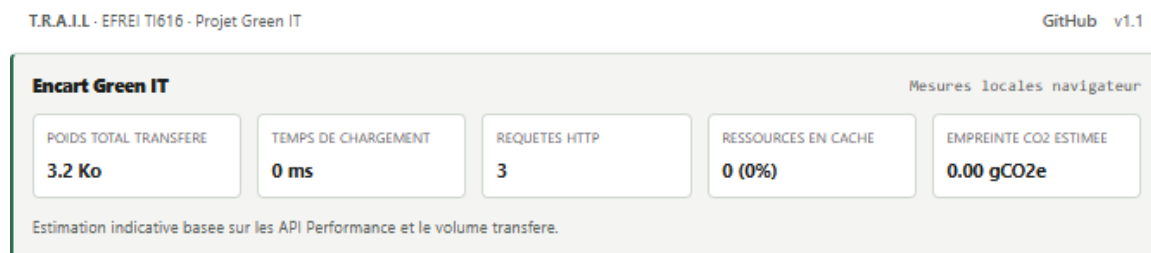


FIGURE 9 – Encart Green IT intégré au pied de page

5.3 Résultat EcoIndex

Une mesure EcoIndex a été réalisée sur la page d’accueil. Le score obtenu montre que l’application atteint un niveau très satisfaisant, notamment grâce à son faible poids et au nombre réduit de requêtes.



FIGURE 10 – Résultat EcoIndex obtenu sur la page d'accueil

Métrique	Objectif	Valeur mesurée
Poids de page	< 200 Ko	0,008 Mo
Requêtes HTTP par page	< 10	3 requêtes
Score EcoIndex	A ou B	A
Score EcoIndex chiffré	—	90 / 100

5.4 Ce qui a été exclu volontairement

Nous avons volontairement évité les cartes interactives intégrées, les notifications push, les publicités, les trackers, les widgets sociaux et les médias lourds. Ces éléments peuvent être utiles dans certains contextes, mais ils auraient alourdi l'application et augmenté son empreinte.

6. Tests et validation

6.1 Scénarios de tests fonctionnels

ID	Scénario	Résultat attendu	Statut
T-01	Créer un compte	Compte créé, redirection correcte	OK
T-02	Se connecter	Session ouverte	OK
T-03	Se déconnecter	Session fermée	OK
T-04	Ajouter un sentier	Sentier visible dans la liste	OK
T-05	Ouvrir le détail d'un sentier	Informations et rapports visibles	OK
T-06	Déposer un rapport	Rapport affiché sur le sentier	OK
T-07	Modifier son profil	Données mises à jour	OK
T-08	Accès non connecté à une route privée	Redirection vers connexion	OK
T-09	Vérifier l'expiration des rapports	Rapport expiré non priorisé	OK

6.2 Validation visuelle

Les différentes pages principales ont été testées manuellement : accueil, liste des sentiers, détail d'un sentier, formulaire de rapport et profil utilisateur. L'objectif était de vérifier que l'application reste lisible, légère et cohérente en navigation.

6.3 Résultats Lighthouse

Afin de valider les performances de l'application, nous avons réalisé plusieurs tests Lighthouse sur les pages principales. Les résultats obtenus montrent que l'application reste très légère et rapide, notamment grâce au rendu serveur, au faible usage de JavaScript et à l'absence de dépendances front-end lourdes.

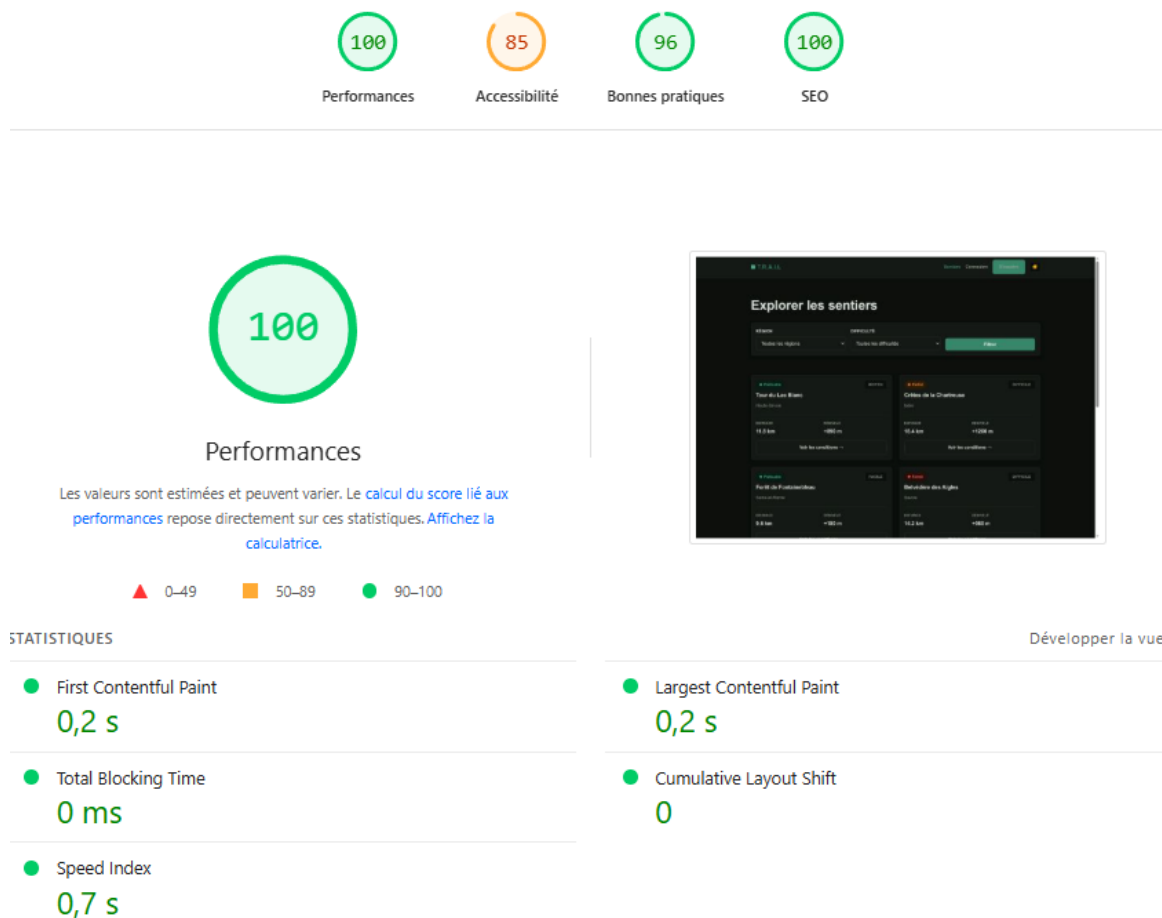


FIGURE 11 — Résultat Lighthouse de la page liste des sentiers

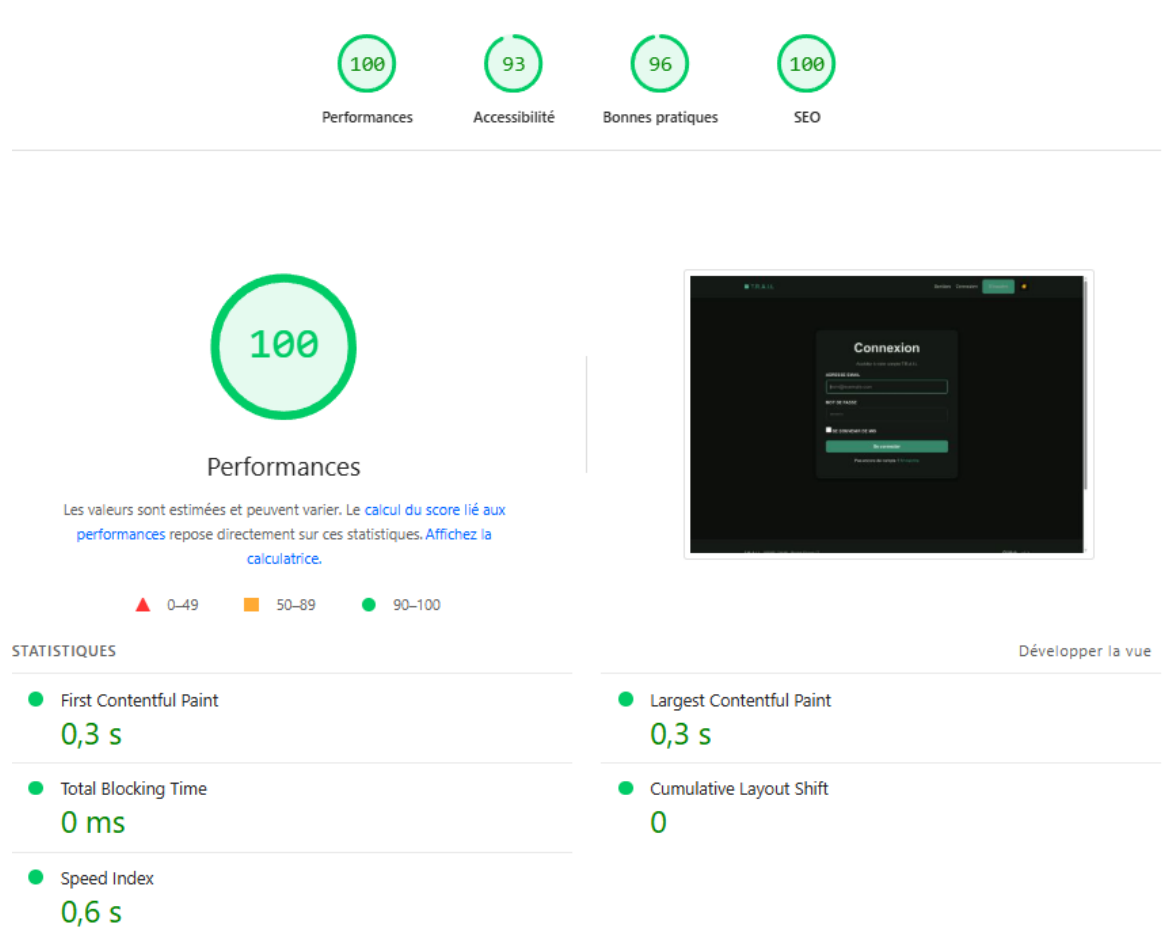


FIGURE 12 – Résultat Lighthouse de la page connexion

Ces résultats confirment que les pages testées atteignent un score de performance de 100, avec des temps de chargement très faibles et aucun blocage important côté navigateur.

7. Organisation de l'équipe et collaboration

7.1 Répartition des tâches

Membre	Responsabilités principales
Loïc Anglard	Développeur - Architecture Flask, backend, authentification, intégration
Esteban Batard-Plaza	Interface, frontend, cohérence graphique
Lilian Cao	Tests, intégration, validation fonctionnelle
Maelle Chollet	Analyse carbone, documentation Green IT
Mathieu Danthine	Développement, support backend et fonctionnalités

7.2 Collaboration

Le projet a été développé de manière itérative. Nous avons d'abord cadré le besoin et le MVP, puis construit progressivement l'application autour des fonctionnalités principales : utilisateurs, sentiers et rapports. Les choix techniques ont été discutés en fonction des contraintes du projet, notamment la sobriété, la simplicité et le temps disponible.

8. Discussion et conclusion

8.1 Ce qui a bien fonctionné

L'architecture Flask avec Blueprints s'est révélée claire et adaptée. Chaque partie du projet est séparée dans un fichier dédié, ce qui rend le code plus simple à lire et à maintenir. Le choix d'un frontend léger a également permis d'obtenir de bons résultats en performance.

8.2 Difficultés rencontrées

La principale difficulté a été la gestion du déploiement en ligne et de la persistance des données. SQLite fonctionne très bien localement, mais il n'est pas adapté à un environnement serverless comme Vercel. La connexion à une base externe PostgreSQL via Supabase a demandé des ajustements supplémentaires.

8.3 Compromis entre fonctionnalités et sobriété

Nous avons dû faire des choix. Par exemple, une carte interactive aurait pu améliorer l'expérience utilisateur, mais elle aurait aussi ajouté des scripts, des ressources externes et une dépendance supplémentaire. Nous avons donc préféré garder une application plus légère et plus simple.

8.4 Pistes d'amélioration

1. Stabiliser complètement la base externe PostgreSQL pour la production.
2. Ajouter un lien externe vers OpenStreetMap plutôt qu'une carte intégrée.
3. Compléter les tests automatisés.
4. Ajouter une purge automatique des rapports expirés.
5. Améliorer les mesures carbone avec plusieurs outils.

8.5 Conclusion

T.R.A.I.L répond au besoin initial : fournir une information récente et utile sur l'état des sentiers. L'application permet de consulter des sentiers, de contribuer à la communauté et de limiter l'affichage de données obsolètes grâce à l'expiration automatique des rapports.

Ce projet nous a aussi montré que le numérique durable ne consiste pas seulement à mesurer un score carbone. Il s'agit surtout de faire des choix dès la conception : limiter les dépendances, éviter les fonctionnalités inutiles, réduire les données stockées et privilégier la simplicité.

La sobriété numérique nous a poussés à construire une application plus simple, plus lisible et plus facile à maintenir. Dans notre cas, les choix les plus durables sont souvent aussi les choix les plus efficaces techniquement.

A. Backlog complet

ID	Fonctionnalité	Priorité	Difficulté	Statut
AU-01	Inscription avec hash bcrypt	Haute	F	OK
AU-02	Connexion avec session Flask-Login	Haute	F	OK
AU-03	Déconnexion	Haute	F	OK
AU-04	Page profil	Moyenne	F	OK
AU-05	Modification du profil	Moyenne	F	OK
SE-01	Liste sentiers avec filtres	Haute	M	OK
SE-02	Page détail sentier	Haute	M	OK
SE-03	Formulaire ajout sentier	Haute	M	OK
RA-01	Formulaire dépôt rapport	Haute	M	OK
RA-02	Expiration automatique	Haute	F	OK
GR-01	Interface légère et responsive	Haute	M	OK
GR-02	Encart Green IT	Moyenne	F	OK
GR-03	Mesures EcoIndex	Moyenne	F	OK

B. Schéma SQL simplifié

```

1 CREATE TABLE IF NOT EXISTS user (
2     id INTEGER PRIMARY KEY AUTOINCREMENT,
3     nom VARCHAR(100) NOT NULL,
4     email VARCHAR(150) UNIQUE NOT NULL,
5     mdp_hash VARCHAR(255) NOT NULL,
6     niveau TEXT CHECK(niveau IN ('d butant', 'intermédiaire', 'expert')),
7     localisation VARCHAR(100),
8     is_admin BOOLEAN DEFAULT 0,
9     date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP
10 );
11
12 CREATE TABLE IF NOT EXISTS sentier (
13     id INTEGER PRIMARY KEY AUTOINCREMENT,
14     nom VARCHAR(150) NOT NULL,
15     region VARCHAR(100) NOT NULL,
16     distance_km REAL NOT NULL CHECK(distance_km > 0),
17     denivele_pos INTEGER NOT NULL CHECK(denivele_pos >= 0),
18     difficulte TEXT NOT NULL,
19     types_pratique TEXT NOT NULL DEFAULT 'trail',
20     terrain VARCHAR(100),
21     saison_recommandee VARCHAR(100),
22     description TEXT,
23     user_id INTEGER NOT NULL,
24     date_ajout DATETIME DEFAULT CURRENT_TIMESTAMP,
25     FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
26 );
27
28 CREATE TABLE IF NOT EXISTS rapport (
29     id INTEGER PRIMARY KEY AUTOINCREMENT,
30     user_id INTEGER NOT NULL,
31     sentier_id INTEGER NOT NULL,
32     date_rapport DATETIME DEFAULT CURRENT_TIMESTAMP,
33     date_expiration DATETIME NOT NULL,
34     statut TEXT NOT NULL,
35     type_pratique TEXT NOT NULL,
36     obstacles TEXT,
37     commentaire TEXT,
38     FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE,
39     FOREIGN KEY (sentier_id) REFERENCES sentier(id) ON DELETE CASCADE
40 );

```

Listing 5 – Schéma SQLite simplifié — T.R.A.I.L

C. Captures complémentaires



FIGURE 13 – Capture complète de la page d'accueil

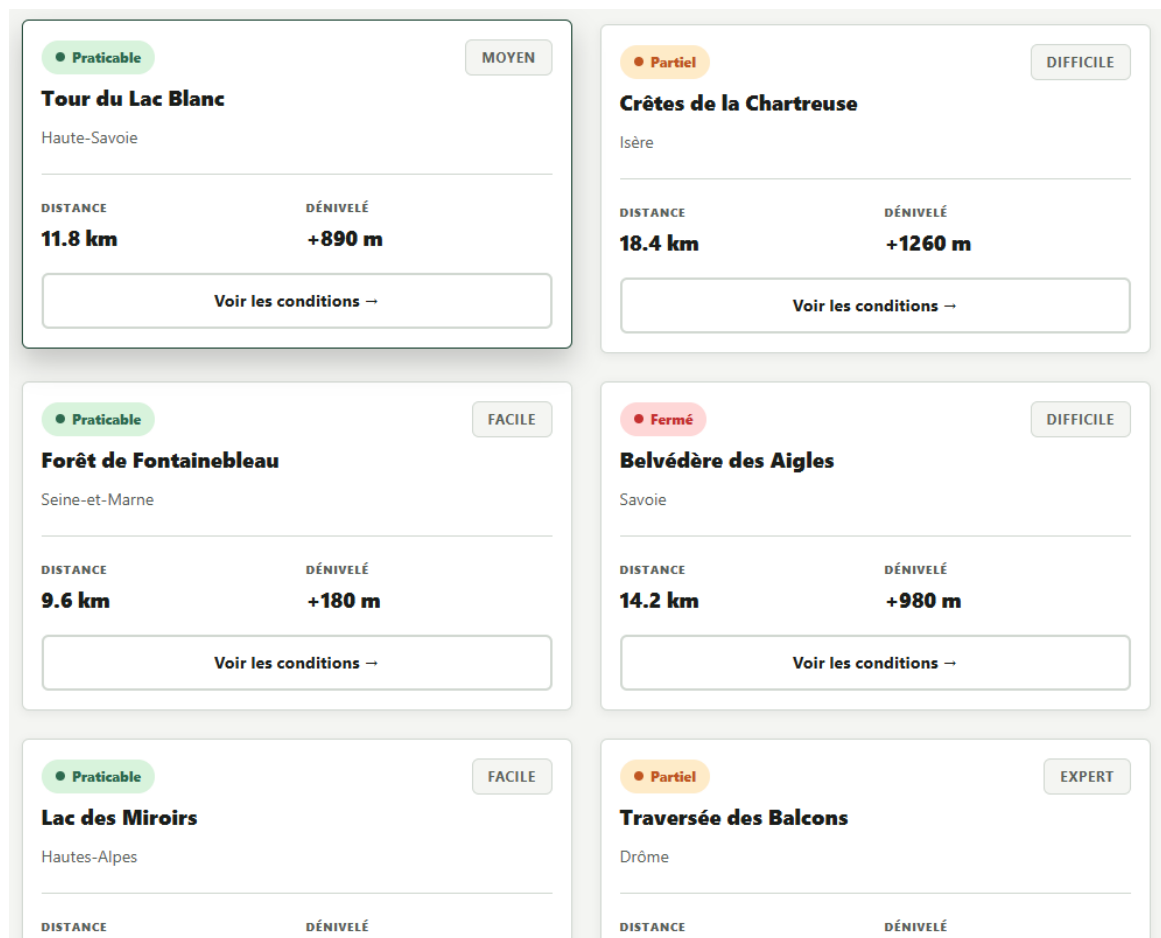


FIGURE 14 – Capture complète de la liste des sentiers

[← Retour à l'exploration](#)

[Signaler l'état](#)[Modifier](#)[Supprimer](#)

Tour du Lac Blanc

Haute-Savoie · Ajouté par @Alice Martin

DISTANCE

11.8 km

DÉNIVELÉ POSITIF

+890 m

NIVEAU DE DIFFICULTÉ

MOYEN

Informations techniques

Type de terrain

Période conseillée

Activités adaptées

Alpin

Juin à Octobre

TRAIL

RANDO

Description

Boucle panoramique très fréquentée.

Rapports de la communauté

2 rapport(s) actif(s)

● Praticable

TRAIL

@Alice Martin

2026-05-06

OBSTACLES SIGNALÉS : Sec · Propre

"Très bon état, quelques portions humides au lever du jour."

Modifier

Supprimer

● Partiel

TRAIL

@Nadia Laurent

2026-05-02

OBSTACLES SIGNALÉS : n...

FIGURE 15 — Capture complète de la fiche d'un sentier

Déposer un rapport

Partagez l'état du terrain avec la communauté

SENTIER CONCERNÉ *

Tour du Lac Blanc (Haute-Savoie) ▼

ÉTAT ACTUEL DU SENTIER *

☒ **PRATICABLE** CONDITIONS IDÉALES

☐ **PARTIEL** PASSAGES DÉLICATS

☐ **FERMÉ** DANGERS / TRAVAUX

VOTRE ACTIVITÉ *

Trail ▼

OBSTACLES RENCONTRÉS

☐ NEIGE ☐ BOUE

☐ VERGLAS ☐ ARBRE TOMBE

☐ CRUE ☐ TRAVAUX

☐ AUTRE

PRÉCISIONS ET OBSERVATIONS

Ex: Neige à partir de 1500m, arbres en travers du sentier sur la portion haute...

Ce signalement sera public et expirera automatiquement après 7 jours pour garantir la fraîcheur de l'information.

Déposer le rapport **Annuler**

FIGURE 16 — Capture complète du formulaire de dépôt de rapport

Mon profil — @Alice Martin

Inscrit le 2026-05-06

Modifier mes informations

NOM

NIVEAU

LOCALISATION

Laisser vide pour garder le mot de passe actuel

NOUVEAU MOT DE PASSE

CONFIRMER

Enregistrer

Actions de compte

[Se déconnecter](#) [Supprimer le compte](#)

Mes rapports récents

praticable	- 2026-05-06
ferme	- 2026-05-06
praticable	- 2026-05-03

Mes sentiers

Tour du Lac Blanc — Haute-Savoie

Belvédère des Aigles — Savoie

[+ Ajouter un sentier](#)

FIGURE 17 — Capture complète de la page profil